BÖLÜM 1 / MODULE 1

Büyük Tersine Dönüş (The Inversion) & AI Çağında Use Case Rönesansı

The AI Inversion & Why Use Cases Rule the GenAI Era

Yazılım mühendisliği tarihinin en büyük ekonomik kırılması: Kodlama ucuzladı, niyet (intent) paha biçilmez oldu. Vibe coding'in çöküşü, determinizm yanılgısı, 1987'den günümüze Use Case evrimi ve Spec-Driven Development (SDD) paradigması.

The greatest economic shift in software history: Implementation is cheap, intent is scarce. The failure of vibe coding, the determinism fallacy, the evolution of use cases from 1987 to 2026, and the Spec-Driven Development paradigm.

1

BÖLÜM: BÜYÜK TERSINE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

Büyük Tersine Dönüş (The Inversion): Kod Ucuzladı, İfade Edilmiş Niyet Kıymete Bindi

The Great Inversion: Implementation is Cheap, Intent is Scarce

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                YAZILIM EKONOMİSİNDE BÜYÜK KIRILMA (THE INVERSION)                │
└──────────────────────────────────────────────────────────────┘

  1. GELENEKSEL DÜNYA (1970 - 2022):
     [ Niyet / Spesifikasyon: UCUZ LAF ] ──► [ Kodlama: ÇOK PAHALI, YAVAŞ, KIT ]
     * Geliştiriciler koridor sohbetlerinde boşlukları sezgileriyle doldururdu.
     * "Kod tek gerçektir" (Code is the truth) demek bir sinizm değil, ekonomik zorunluluktu.

  2. ÜRETKEN YAPAY ZEKA ÇAĞI (2023 - 2026+):
     [ Niyet / Spesifikasyon: KIT & PAHA BİÇİLMEZ ] ──► [ Kodlama: UCUZ, IŞIK HIZINDA ]
     * Kod artık bir kaynak değil; derleyici çıktısı gibi üretilip atılabilen bir ara üründür!
     * AI her niyet boşluğunu kendi uydurduğu (halüsinasyon) sahte bir özellikle doldurur!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eskiden bir heykeltıraşa mermerden heykel yaptırmak aylar sürerdi ve çok pahalıydı; ona kabataslak 'Güzel bir at yap' desen bile adam oturup günlerce yontar, eksikleri zanaatiyle tamamlardı. Bugün ise elinde saniyede 10.000 heykel basabilen sihirli bir 3D yazıcı (Yapay Zekâ) var! Ama yazıcıya sadece 'at yap' dersen, sana kanatlı, 6 bacaklı, yeşil bir canavar basar ve 'Sen tam olarak ne istediğini söylemedin ki!' der. Artık heykel yontmak bedava; paha biçilmez olan şey heykelin milimetrik çizimidir!

🇬🇧 ENGLISH (ELI5)

Historically, commissioning a stone sculptor was exorbitant and slow. If you gave vague instructions like 'carve a horse', the master craftsman filled the gaps with personal aesthetic judgment. Today, generative AI is a hyper-speed 3D printer that manufactures statues in 3 seconds for pennies. But if you lack precise blueprints, the printer invents a 6-legged neon unicorn and presents it with total confidence. Carving stone is now free; the rare, precious asset is the architectural blueprint of your intent.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Simon Martinelli'nin 'Writing Use Cases for AI' kitabında ortaya koyduğu 'The Inversion' tezi: Yazılım tarihi boyunca kod yazmak (implementation) pahalı, şartname hazırlamak (specification) ucuz bir formaliteydi. Ekipler eksik şartname yazmayı göze alabiliyordu çünkü kıdemli mühendisler eksik iş kurallarını kahve molalarında paydaşlarla konuşup sezgileriyle kapatıyordu. LLM'lerin (Claude Code, Cursor, Antigravity) gelmesiyle denklem tersyüz oldu. Yapay zekâ ajanı belirsiz bir prompt aldığında soru sormaz; boşluğu en olası gördüğü varsayımla sessizce doldurur. Sonuç: İlk bakışta çalışan ama asla sürdürülemeyen, yeniden üretilemeyen (regenerate edilemeyen) 'vibe coding spagettisi'. Çözüm: Şartnameyi birincil varlık (asset), kodu ise derlenebilir bir çıktı (output) kabul etmektir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The core thesis of 'The Inversion': Throughout software engineering history, implementation was bottlenecked by expensive human labor, while specifications were treated as disposable scaffolding. Teams could afford under-specification because human engineers exercised contextual judgment during hallway discussions. Generative AI fundamentally inverts this economic equation: code generation is near-zero marginal cost, deterministic, and instantaneous. What is scarce is expressed, testable intent. When prompted with ambiguous requirements, an AI agent does not pause to ask for clarification; it hallucinates a plausible feature set that compiles but diverges from business reality.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Niyetin Açıkça Belirtildiği Use Case Arayüzü (Contract First)
package com.maang.aiup.banking;

public class LoanApplicationUseCase {
    // Niyet: Tam ve kesin girdi modeli (Records ile değişmezlik)
    public record ApplyLoanCommand(String memberId, double amountUsd, int durationMonths) {
        public ApplyLoanCommand {
            if (amountUsd <= 0) throw new IllegalArgumentException("Kredi tutarı pozitif olmalıdır.");
            if (durationMonths < 1 || durationMonths > 60) throw new IllegalArgumentException("Vade
1-60 ay arasında olmalıdır.");
        }
    }

    public record LoanDecision(boolean approved, String loanId, double interestRate, String
rejectionReason) {}

    // Sistem Sözleşmesi: AI'ın kafasına göre kural uydurmasını engelleyen kesin metod
    public interface LoanService {
        LoanDecision applyForLoan(ApplyLoanCommand command);
    }

    void main() {
        var cmd = new ApplyLoanCommand("MEM-8921", 50000.0, 24);
        System.out.println("Niyet açıkça modellendi: " + cmd);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Vibe coding tuzağı: Prompt ile hızlıca ayağa kaldırılan kodlar ilk 2 gün harika görünür; ancak 3. hafta bir iş kuralı değiştiğinde kodun neden öyle yazıldığını kimse bilemez ve yeniden üretilemez. Şartnamesiz üretilen kod teknik borç değil, teknik iflastır!

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

The Vibe Coding Trap: Generating software from loose prompts produces code that works on day one but is unmaintainable by day ten. Without a versioned specification, the business intent lives only in ephemeral chat history, rendering automated regeneration impossible.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Kodlama artık ucuz ve tekrarlanabilir bir emtiadır; asıl kıymetli varlık açıkça ifade edilmiş niyettir.
- AI belirsizlik gördüğünde durup soru sormaz; boşluğu uydurma (halüsinasyon) özelliklerle doldurur.
- AI Unified Process'te şartname ana varlıktır, kod ise onun derlenebilir bir çıktısıdır.

🎯 Key Architectural Takeaways

- Implementation is now a cheap commodity; precisely expressed intent is the primary asset.
- AI never pauses at ambiguities; it fills gaps with plausible hallucinations.
- In AIUP, the specification is the asset, and code is merely its generated output.

2

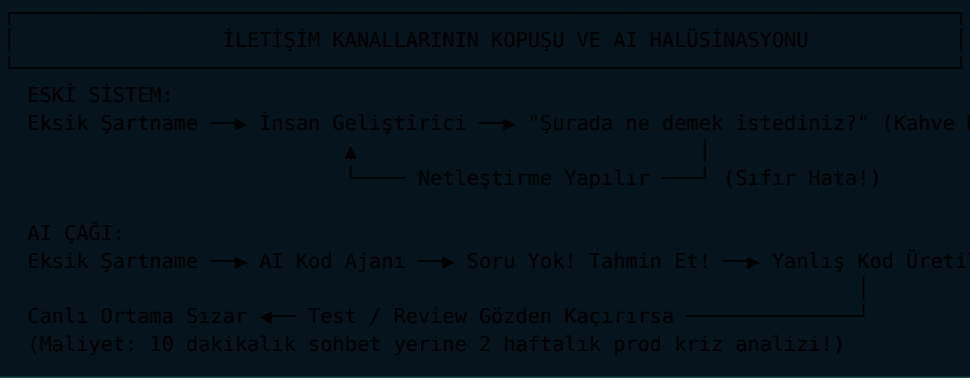
BÖLÜM: BÜYÜK TERSİNE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

Koridor Sohbeti Öldü: İletişim Boşluklarının ve AI Halüsinasyonunun Gerçek Maliyeti

The Hallway Conversation is Dead: The True Cost of Code Hallucination

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski iş yerlerinde stajyer bir çocuğa 'Müşterilere indirim yap' dediğinde, çocuk masana gelip 'Abi kime yüzde kaç indirim yapacağız?' diye sorardı. Yapay zekâ ise soru sormayan, aşırı özgüvenli bir robottur. Ona 'indirim yap' dersen, kafasına göre şirketin en pahalı ürününü herkese %90 indirimle satar ve sana 'İndirim yaptım, harika oldu!' diye gülümser. Soru soracak insan kalmadığı için kuralları en baştan taşa kazımak zorundasın!

🇬🇧 ENGLISH (ELI5)

In a traditional office, if you told an apprentice engineer to 'discount the shopping cart', they would walk to your desk and clarify the percentage rules. An AI coding agent is a hyper-confident worker that never asks questions. Tell it to 'discount carts', and it might quietly slash prices by 90% across the entire inventory, presenting the pull request with a cheerful smile. Because hallway questions are gone, precision must be baked into the spec upfront.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Geleneksel yazılım süreçlerinde 'Eksik Belgeleme' (Under-specification) bir felaket değildi çünkü organizasyonun içinde 'Sosyal Netleştirme Ağı' (Social Disambiguation Network) vardı: Slack mesajları, standup toplantıları ve koridor sohbetleri. Bir junior mühendis şartnamede bir çelişki gördüğünde işi durdurup sorardı; bu soru ücretsiz bir 'belirsizlik dedektörü' görevi görürdü. AI ajanları (Claude, Copilot, Antigravity) ise bu dedektörü tamamen yok etti. Bir ajan, boş bırakılan her alanı istatistiksel ağırlıklarına göre en mantıklı gelen kod parçasıyla tamamlar. Martinelli'nin belirttiği gibi: Eskiden belirsizliğin maliyeti 5 dakikalık bir sohbetti; bugün ise canlıya çıkan ve haftalarca fark edilmeyen sessiz veri bozulmalarıdır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

In classical workflows, under-specification was buffered by organizational social networks: hallway chats, quick Slack pings, and standup debates. An observant human developer served as a free ambiguity detector: encountering an unhandled edge case, they halted execution and sought clarification. LLM-based agents eradicate this feedback loop. They suffer from zero hesitation, synthesizing plausible implementations across unstated boundaries. The cost of ambiguity has migrated from a 5-minute conversation to production data corruption that escapes unnoticed until an audit.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Boşluk Bırakmayan Domain Doğrulama Mekanizması
package com.maang.aiup.discount;

import java.math.BigDecimal;

public class DiscountPolicy {
    // İş Kuralı BR-001: İndirim sadece VIP üyelere ve maksimum %20 uygulanabilir.
    public record DiscountRequest(String customerTier, BigDecimal cartTotal, BigDecimal
requestedDiscountPercent) {}

    public record ValidatedDiscount(BigDecimal finalDiscountAmount, String appliedRule) {}

    // AI'ın kafasına göre indirim uydurmasını engelleyen deterministik kural motoru
    public static ValidatedDiscount calculate(DiscountRequest req) {
        if (!"VIP".equalsIgnoreCase(req.customerTier())) {
            return new ValidatedDiscount(BigDecimal.ZERO, "BR-001: Standart üyelere indirim
uygulanamaz.");
        }
        // AI'ın %90 yapmasını engelleyen sınır
        BigDecimal maxAllowed = new BigDecimal("20.00");
        BigDecimal effectivePercent = req.requestedDiscountPercent().min(maxAllowed);
        BigDecimal discount = req.cartTotal().multiply(effectivePercent).divide(new
BigDecimal("100"));

        return new ValidatedDiscount(discount, "BR-001: VIP indirimi başarıyla hesaplandı.");
    }

    void main() {
        var req = new DiscountRequest("VIP", new BigDecimal("1000.00"), new BigDecimal("35.00"));
        var result = calculate(req);
        System.out.println("Sonuç: " + result);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

AI ile çalışırken 'Sistem kullanıcıyı uygun şekilde bilgilendirir' veya 'Gereken indirim yapılır' gibi yuvarlak cümleler kurmayın! 'Uygun şekilde' kelimesi, sistemin mimarisini ve kurallarını tamamen yapay zekanın hayal gücüne terk etmek demektir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never write vague requirements like 'system notifies user appropriately' or 'applies a reasonable discount'. The phrase 'appropriately' delegates total architectural authority to an algorithm that cannot feel business consequences.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- İnsan geliştiricilerin sorduğu netleştirme soruları ücretsiz bir belirsizlik dedektörüydü; AI bu dedektörü yok etti.
- Belirsizlik artık bir soru olarak geri dönmez; yanlış ve sessizce çalışan bir kod olarak canlıya sızar.
- Tüm iş kuralları ve limitler şartnamede sayısal ve net olarak tanımlanmalıdır.

🎯 Key Architectural Takeaways

- Human clarifying questions were free ambiguity detectors; AI agents completely eliminate them.
- Ambiguity no longer surfaces as a question; it manifests as silent production bugs.
- All business invariants and limits must be specified with numerical precision in the spec.

3

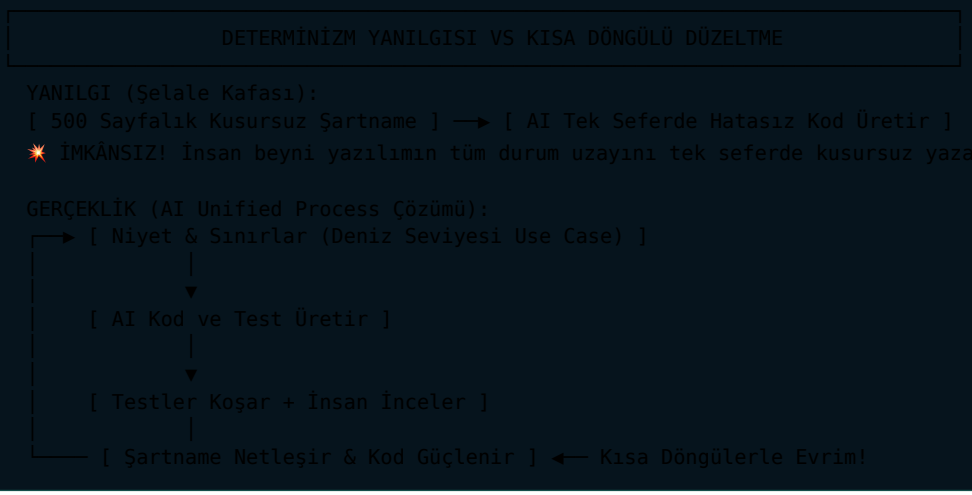
BÖLÜM: BÜYÜK TERSINE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

Determinizm Yanılgısı: Mükemmel Şartname Diye Bir Şey Yoktur ve Gerek de Yoktur!

The Determinism Fallacy: Why Perfect Specs Don't Exist and Aren't Needed

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bazı insanlar 'Yapay zekanın doğru kod yazması için 10.000 sayfalık kusursuz bir kural kitabı yazmalıyız, bu da imkansız, demek ki AI bir işe yaramaz!' der. Bu şuna benzer: 'Bir şoföre gideceği yeri anlatmak için yoldaki her taşın yerini tarif edemem, o halde taksiye binmeyeyim!' Hayır! Şoföre sadece 'Kadıköy rıhtıma git, sahilden sür' dersin (sınırlar). Yanlış sokağa girerse 'Sağa dön' dersin (kısa döngülü test). Mükemmel tarif gerekmez; net hedef ve anında düzeltme yeterlidir!

🇬🇧 ENGLISH (ELI5)

Skeptics argue: 'AI generation only works if the spec is 100% mathematically complete to force deterministic code. Perfect specs are impossible, therefore AI coding is a failure.' This is like refusing to take a taxi because you cannot describe the molecular coordinates of every pebble on the highway. You don't need a perfect spec; you need clear boundary constraints and rapid automated tests that catch deviations within seconds.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Determinizm Yanılgısı (The Determinism Fallacy): Birçok mühendis, AI güdümlü yazılım geliştirmenin ancak 'tamamen deterministik ve sıfır boşluklu' devasa spesifikasyonlarla mümkün olabileceğini savunarak yöntemi baştan reddeder. Cockburn ve Martinelli bu tezi çürütür: 1) İlk taslak değerin %50'sini üretir; kelimeler üzerinde sonsuza kadar oynamak iletişimi artırmaz. 2) Amaç deterministik bir robot yaratmak değil, 'her okuyucunun (insan paydaş, mühendis ve AI) aynı niyeti anladığı' bir zemin kurmaktır. 3) Kalite, tek seferlik kusursuz yazımda değil; Şartname -> Üretim -> Test Doğrulaması -> Paydaş İncelemesi döngüsünün dakikalar içinde tekrarlanabilmesinde saklıdır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Determinism Fallacy deconstructed: The objection states that generative AI requires airtight, mathematically exhaustive specifications to prevent non-deterministic code branching. Because perfect specifications are impossible in human language, the approach is deemed unviable. Martinelli refutes this: the AI Unified Process never demands exhaustive determinism. It requires precision strictly about observable behavior, backed by immutable test assertions. Quality is achieved not through monolithic upfront perfection, but through hyper-short feedback loops: specify intent, generate, verify with unit/slice tests, refine.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Kısa Döngülü Test Doğrulaması ile Niyet Denetimi (Test-Protected)
package com.maang.aiup.verification;

public class OrderValidationEngine {
    public record OrderItem(String sku, int quantity, double unitPrice) {}
    public record Order(String orderId, java.util.List<OrderItem> items) {}

    // Kural BR-010: Bir siparişte en fazla 10 adet ürün olabilir ve toplam tutar pozitif olmalıdır.
    public static boolean validateOrder(Order order) {
        if (order.items() == null || order.items().isEmpty()) return false;
        int totalQty = order.items().stream().mapToInt(OrderItem::quantity).sum();
        if (totalQty > 10) return false;
        double total = order.items().stream().mapToDouble(i -> i.quantity() * i.unitPrice()).sum();
        return total > 0.0;
    }

    void main() {
        // Hızlı doğrulama testi (Determinizmi koruyan emniyet kemeri)
        var validOrder = new Order("ORD-1", java.util.List.of(new OrderItem("BOOK-1", 2, 25.0)));
        var invalidOrder = new Order("ORD-2", java.util.List.of(new OrderItem("BOOK-2", 15, 10.0)));

        assert validateOrder(validOrder) : "Geçerli sipariş onaylanmalıydı!";
        assert !validateOrder(invalidOrder) : "10 adetten fazla ürün reddedilmeliydi!";

        System.out.println("✅ Determinizm testlerle koruma altına alındı!");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnameyi kusursuzlaştırmak için haftalarca toplantı yapıp tek satır kod üretmemek (Analysis Paralysis) şelale modeline geri dönüştür. İlk gün deniz seviyesi taslağı çıkarın, AI'a ürettirin ve test sonuçlarına bakarak şartnameyi rafine edin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Spending weeks polishing a specification without running an AI generation pass is a regression into the waterfall trap. Draft the sea-level spec in hours, run `/implement`, inspect the generated test suite, and iterate.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Mükemmel şartname imkânsızdır ve gerekli de değildir; önemli olan şey gözlemlenebilir davranışın netliğidir.
- Determinizm tek seferlik yazımla değil, testlerle korunan kısa çevrimli geri bildirim döngüleriyle sağlanır.
- Şartname niyetin sınırlarını çizer, AI bu sınırlar içinde kodu doldurur, testler ise sınır ihlalini yakalar.

🎯 Key Architectural Takeaways

- Perfect specifications are impossible and unnecessary; observable clarity is what matters.
- Determinism is enforced by automated test harnesses in rapid iterative loops.
- The spec sets the boundaries, AI implements inside them, and tests catch boundary violations.

4

BÖLÜM: BÜYÜK TERSİNE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

Use Case'in 40 Yıllık Evrimi: Ericsson (1987), Cockburn (2001) ve Use-Case 3.0 (2024)

The Evolution of Use Cases: From Ericsson (1987) to Cockburn (2001) to Use-Case 3.0

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                    USE CASE'İN 40 YILLIK BÜYÜK YOLCULUĞU

1987: Ivar Jacobson (Ericsson)
└──► OOPSLA bildirisi: Telefon santralleri için kullanıcı odaklı senaryolar.
     "Sistem ne yapar?" sorusunun ilk yapısal cevabı.

2001: Alistair Cockburn ("Writing Effective Use Cases")
└──► Yazma sanatı: Hedef Seviyeleri (Deniz/Bulut/Balık), Paydaş Çıkarları,
     Ana Akış ve Alternatif Akış disiplini.

2000-2020: Çevik Yozlaşma (User Stories İstismarı)
└──► Post-it çılgınlığı! Parçalanmış, bağlamsız, uç durumları unutan yapışkan kağıtlar.

2024: Jacobson & Cockburn ("Use-Case 3.0")
└──► Dilimleme (Slices), Çevik ve Hafif Entegrasyon: "Use Cases are Essential!"

2026+: AI Unified Process & Simon Martinelli
└──► Ajanlar için yeniden doğuş: AI'ın anlayabileceği tek tam davranışsal kontrat!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

1980'lerde icat edilen çekiç ve tekerlek gibi bazı aletler o kadar mükemmeldir ki teknoloji ne kadar değişirse değişsin eskimez. 1987'de telefon santralleri için icat edilen 'Use Case', 2000'lerde 'Biz artık çeviğiz, post-it yapıştırıyoruz' denilerek çöpe atılmak istendi. Ama yapay zekâ gelince görüldü ki, mini minnacık post-it notlarıyla koca sistemleri AI'a anlatamazsın! 40 yıllık Use Case, yapay zekanın en iyi anlayabildiği süper modern bir mimari haritaya dönüştü!

🇬🇧 ENGLISH (ELI5)

Fundamental inventions like the wheel or the hammer never become obsolete. Invented in 1987 by Ivar Jacobson at Ericsson and formalized by Alistair Cockburn in 2001, use cases were temporarily sidelined by agile teams wielding single-sentence sticky notes. But generative AI cannot build complex systems from disconnected sticky notes. Today, Use-Case 3.0 and the AI Unified Process have restored use cases as the premier behavioral contract for autonomous agents.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Use Case'in tarihsel kırılma noktaları: 1) 1987 Jacobson OOPSLA: Nesne yönelimli programlamanın (OOP) içine 'davranışsal hikaye' kavramını soktu; Rational Unified Process'in (RUP) üç temel direğinden biri oldu (Use-Case Driven). 2) 2001 Cockburn: Şablon çılgınlığını sadeleştirdi; Deniz Seviyesi (Sea Level) hedefleri ve başarısızlık durumlarını garanti altına alan sözleşme (contract) modelini oturttu. 3) 2023-2024 ACM Queue & Use-Case 3.0: Jacobson ve Cockburn 'Use Cases are Essential' makalesiyle kullanım senaryolarının çevik dilimlerle (slices) Scrum ve Kanban'a entegre olabileceğini kanıtladı. 4) 2026 AIUP: Martinelli, üçüncü okuyucunun (AI Ajanı) gelişiyle use case'in kod ve test üreten birincil generatif varlık olduğunu ilan etti.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The 40-year architectural lineage: 1) 1987 Jacobson (Ericsson): Introduced use cases at OOPSLA to model telecommunication switching flows, establishing the core axiom: software must be driven by user goals. 2) 2001 Cockburn: Standardized the craft in 'Writing Effective Use Cases', defining goal altitude tiers (Cloud, Sea, Fish), stakeholder guarantees, and the 3-kind step hierarchy. 3) 2024 Use-Case 3.0: Reconciled use cases with lightweight agile delivery via 'use-case slices'—modular delivery units threading through the basic flow into edge paths. 4) 2026 AIUP (Martinelli): Positioned use cases as the central generative hub connecting system vision to autonomous AI coding agents.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Use-Case 3.0 Dilimleme (Slice) Mantığının Kodda Karşılığı
package com.maang.aiup.slices;

public class BookBorrowingSlice {
    // Dilim 1 (Slice 1): Sadece Temel Başarı Senaryosu (Happy Path)
    public record BorrowBasicSlice(String memberId, String bookIsbn) {
        public void execute() {
            System.out.println("Slice 1 Çalıştı: Kitap ödünç verildi, teslim tarihi 28 gün sonrası.");
        }
    }

    // Dilim 2 (Slice 2): Alternatif Akış Eklenmiş Gelişmiş Dilim
    public record BorrowWithLimitCheckSlice(String memberId, String bookIsbn, int activeLoanCount) {
        public boolean execute() {
            if (activeLoanCount >= 5) {
                System.out.println("Slice 2 (A2 Flow): Limit aşıldı! Ödünç engellendi.");
                return false;
            }
            System.out.println("Slice 2: Kontrol başarılı, kitap verildi.");
            return true;
        }
    }

    void main() {
        new BorrowBasicSlice("M101", "ISBN-9988").execute();
        new BorrowWithLimitCheckSlice("M101", "ISBN-9988", 5).execute();
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Geçmişte RUP döneminde yazılan 100 sayfalık bürokratik use case dökümanlarıyla modern Use-Case 3.0'ı karıştırmayın! Modern use case hafiftir, 3-9 adımlı senaryolardan oluşur ve doğrudan dilimler (slices) halinde kodlanır.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not conflate modern AIUP use cases with 1990s RUP bureaucratic monoliths. Modern use cases are lean, bounded to 3-9 steps, sliced iteratively, and engineered for direct machine parsing.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Use case 1987'de doğmuş, 2001'de Cockburn ile olgunlaşmış ve 2024'te Use-Case 3.0 ile çevikleşmiştir.
- Kullanıcı hikayeleri konuşma başlatır; Use Case ise varılan kararların ve kuralların yaşadığı yerdir.
- AI çağında use case, kodlama ajanlarına niyet aktarabilen tek eksiksiz davranış formatıdır.

🎯 Key Architectural Takeaways

- Use cases originated in 1987, matured via Cockburn in 2001, and became agile in 2024.
- User stories initiate conversations; use cases store the architectural decisions.
- In the AI era, use cases are the only established format that conveys complete behavioral context.

5

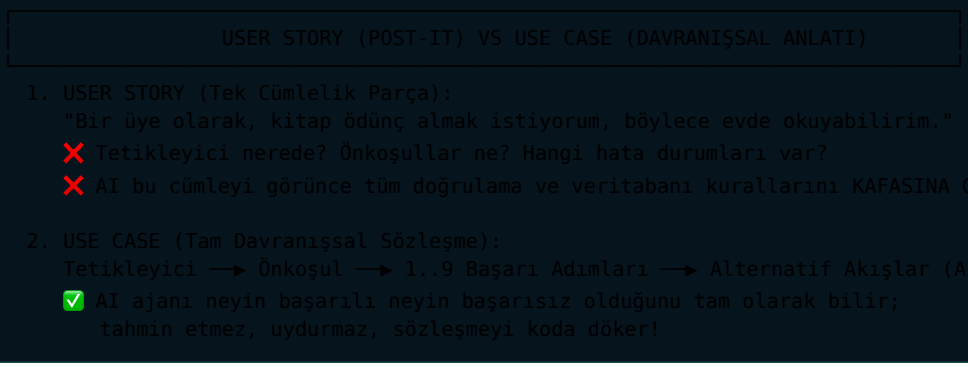
BÖLÜM: BÜYÜK TERSINE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

Kullanıcı Hikayeleri vs Use Case: Neden Yapışkan Post-it Kağıtları LLM Çağında İflas Etti?

User Stories vs Use Cases: Why Agile Sticky Notes Broke in the Age of LLMs

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

User Story, buzdolabının üstüne yapıştırılan 'Akşama ekmek al' notu gibidir. Eve gelen eşin fırının kapalı olduğunu görürse ne yapacağını bilmez (eve boş döner veya pastaneden pasta alır). Use Case ise tam bir görev talimatıdır: 'Akşam fırından 2 tam buğday ekmeği al. Fırın kapalıysa yan bakkala bak. Bakkalda da yoksa beni ara, sakın beyaz ekmek alma!' Yapay zekâya post-it verirsen kafası karışır; Use Case verirsen harfiyen yerine getirir!

🇬🇧 ENGLISH (ELI5)

A user story is a fridge note that says: 'Buy bread on the way home.' If the bakery is locked, your partner is baffled: do they visit a grocery store, buy cake, or return empty-handed? A use case is the full protocol: 'Buy two sourdough loaves at Bakery A. If closed (A1), proceed to Market B. If sold out (A2), call home. Under no circumstances buy white bread.' AI agents flounder with fridge notes; they thrive on structured protocols.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Kullanıcı Hikayeleri (User Stories) ile Use Case arasındaki tamamlayıcı ilişki: Çevik ekipler 20 yıl boyunca use case'lerin yerini user story'lerin aldığını iddia etti. Oysa Cockburn'ün 2024'te vurguladığı gibi: User Story bir konuşma biletidir (token for a conversation); detay içermez. Ancak AI ile çalışırken karşınızda kahve içip sohbet edebileceğiniz bir insan yoktur! Bir LLM ajanına 'As a user I want to borrow a book' prompt'u verirseniz; kaç kitap limiti olduğunu, gecikmiş borç kontrolünü, kitap kopyası kalmadığında ne yapılacağını bilemez ve kafasına göre bir senaryo yazar. AI Unified Process'te `docs/requirements.md` içinde gereksinimler User Story formatında listelenir (FR-012); ancak bu hikayelerin gerçek davranışsal sözleşmesi Use Case dosyalarında (UC-002) yaşar.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The dialectic of User Stories vs Use Cases: User stories ('As a… I want… So that…') were explicitly designed as conversation starters—placeholders for verbal negotiations between product owners and developers. But LLMs cannot engage in informal office banter; they generate code immediately from whatever text they are fed. Feeding an ambiguous user story into an AI coding engine forces the model to invent preconditions, error pathways, and state rollbacks. In the AI Unified Process, user stories express functional requirements in the catalog (`docs/requirements.md`), while use cases encapsulate the multi-path behavioral contract.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: User Story'nin Use Case Sözleşmesine Dönüştürülmesi
package com.maang.aiup.contract;

public class LibraryContract {
    // 1. User Story: "Bir üye olarak kitap ödünç almak istiyorum..." (Gereksinim Kataloğu)
    public record UserStoryRef(String frId, String summary) {}

    // 2. Use Case: Tüm alternatif akışları içeren somut davranış modeli
    public static class BorrowBookUseCase {
        public static final UserStoryRef TRACES_TO = new UserStoryRef("FR-012", "Book Borrowing
Flow");

        public record BorrowResult(boolean success, String message, String dueDate) {}

        public BorrowResult execute(String memberId, String isbn, boolean isOverdue, int currentLoans,
int copiesAvailable) {
            // Alternatif Akış A3: Gecikmiş borç kontrolü
            if (isOverdue) return new BorrowResult(false, "A3: Gecikmiş kitap borcunuz var!", null);
            // Alternatif Akış A2: Limit kontrolü
            if (currentLoans >= 5) return new BorrowResult(false, "A2: Maksimum 5 kitap limitine
ulaştınız!", null);
            // Alternatif Akış A1: Stok kontrolü
            if (copiesAvailable <= 0) return new BorrowResult(false, "A1: Uygun kopya kalmadı,
rezervasyon önerildi.", null);

            // Ana Başarı Akışı
            return new BorrowResult(true, "Kitap başarıyla ödünç verildi.", "2026-11-17");
        }
    }

    void main() {
        var uc = new BorrowBookUseCase();
        var res = uc.execute("M101", "ISBN-123", false, 2, 3);
        System.out.println("Use Case İcrası: " + res);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

User Story ile Use Case'i birbirine düşman sanmak büyük bir hatadır. User Story 'Neden ve Kimin için' olduğunu söyler; Use Case ise 'Sistemin adım adım ne yapacağını ve hangi hataları yöneteceğini' belirler.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not treat user stories and use cases as adversarial paradigms. User stories capture business intent and stakeholder value; use cases specify the exhaustive interaction contract and failure topologies.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- User Story konuşma başlatmak için bir jetondur; Use Case ise varılan kararların yaşadığı evdir.
- Tek satırlık kullanıcı hikayesiyle kod üreten AI, tüm köşe vakaları ve kuralları kafasından uydurur.
- AIUP'ta her Use Case en az bir kullanıcı hikayesine (FR-XXX) mekanik olarak izlenebilir şekilde bağlanır.

🎯 Key Architectural Takeaways

- User stories are tokens for conversation; use cases are where the negotiated details live.
- Feeding single-sentence user stories to an AI forces it to hallucinate business logic.
- In AIUP, every use case traces mechanically to at least one user story in the requirements catalog.

6

BÖLÜM: BÜYÜK TERSINE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

Spec-Driven Development (SDD): Şartnameyi Birincil Varlık Olarak Konumlandırma

Spec-Driven Development: Treating the Specification as the Primary Asset

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              SPEC-DRIVEN DEVELOPMENT (SDD) YAŞAM DÖNGÜSÜ │
└──────────────────────────────────────────────────────────┘

KURAL: Kod asla doğrudan elle değiştirilmez! Değişiklik daima şartnameden başlar!

1. Şartnameyi Güncelle (docs/use_cases/UC-002-borrow-book.md)
          |
          ▼
2. /spec-review ile Tutarlılık Denetimi Yap
          |
          ▼
3. /implement Komutunu Çalıştır ──► AI Kodu ve Testleri Yeniden Üretir!
          |
          ▼
4. Testler Yeşile Döner ──► Canlıya Al! (Sıfır Sapma, %100 İzlenebilirlik!)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir gökdelen inşa ederken elektrik tesisatını beğenmediğinde gidip duvarı kırıp kafana göre kablo çekmezsin. Önce mimari projeyi (şartnameyi) güncellersin, mühendisler onaylar, sonra ustalar gelip plana göre tesisatı döşer. Eğer projeyi güncellemeden duvara kablo çekersen, yarın başka bir usta gelip o kabloyu keser ve bina yanar! SDD tam olarak budur: Önce planı düzelt, işçiler (AI) plana göre binayı saniyeler içinde yenilesin!

🇬🇧 ENGLISH (ELI5)

When building a skyscraper, if you decide to add an electrical outlet, you don't secretly chisel into a load-bearing column and solder wires. You update the architectural blueprints first. Once approved, the construction crew executes the change. Modifying code directly while leaving the specification stale is architectural vandalism; the next automated regeneration will wipe out your undocumented hotfix.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Spec-Driven Development (SDD) ve Kodun Geçiciliği Paradigması: Simon Martinelli'nin 'Spec-Driven Development' kitabında altını çizdiği en temel disiplin: Kod artık kalıcı bir varlık değildir; şartnamenin bir derleme çıktısıdır. Geleneksel yaklaşımlarda kod değiştikçe döküman eskir ve çöp olurdu. SDD bu döngüyü kırar: 1) Tek Doğruluk Kaynağı (Single Source of Truth): Şartname ve kod çeliştiğinde her zaman kod hatalı kabul edilir. 2) Yeniden Üretilebilirlik (Regenerability): Yeni bir framework sürümü çıktığında (örneğin Spring Boot 3'ten 4'e veya Java 21'den 25'e), şartname hiç değişmeden tek bir `/implement` komutuyla tüm codebase modern kütüphanelerle baştan üretilebilir. 3) İzlenebilirlik (Traceability): Canlı sistemdeki her bir satır kod, şartnamedeki bir iş kuralına (BR-XXX) ve use case adımına doğrudan bağlanır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Spec-Driven Development (SDD) architectural discipline: SDD posits that software source code is no longer a durable asset; it is an ephemeral compilation artifact derived from formal specifications. The core tenets: 1) Specification Primacy: Whenever source code deviates from the use case specification, the code is definitively defective by definition. 2) Effortless Regeneration: Modernizing frameworks or porting stacks (e.g. Java to TypeScript, JPA to jOOQ) becomes mechanical: rerun generation commands against unchanged, human-verified specifications. 3) Strict Traceability: Every generated controller, entity validation, and integration test links back to a specific business rule identifier (BR-XXX).

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Spec-Driven Kod Üretiminin İdeal Çıktısı (Tam İzlenebilir Sınıf)
package com.maang.aiup.sdd;

/**
 * ⚠ DİKKAT: Bu sınıf Spec-Driven Development kapsamında AI tarafından üretilmiştir.
 * Bu dosyayı doğrudan elle DEĞİŞTİRMEYİN!
 * Değişiklikler için şartnameyi güncelleyin: docs/use_cases/UC-002-borrow-book.md
 *
 * Traces to:
 * - Use Case: UC-002 (Borrow Book)
 * - Requirements: FR-012, NFR-002
 * - Rules: BR-004 (Availability), BR-005 (Loan Limit), BR-006 (Overdue Block)
 */
public final class BorrowBookService {
    public record LoanConfirmation(String loanId, String memberId, String bookCopyId,
java.time.LocalDate dueDate) {}

    public LoanConfirmation processLoan(String memberId, String bookCopyId, int activeLoans, boolean
hasOverdue, int availableCopies) {
        // Step 4: System validates that a copy is available (BR-004)
        if (availableCopies <= 0) {
            throw new IllegalStateException("BR-004: Kitap kopyası mevcut değil.");
        }
        // Step 5: System validates member has not reached limit (BR-005)
        if (activeLoans >= 5) {
            throw new IllegalStateException("BR-005: Üye 5 olan ödünç limitine ulaştı.");
        }
        // Step 6: System validates member has no overdue loans (BR-006)
        if (hasOverdue) {
            throw new IllegalStateException("BR-006: Gecikmiş borçlar nedeniyle ödünç verilemez.");
        }

        // Step 7: System records loan with due date 28 days from today
        var dueDate = java.time.LocalDate.now().plusDays(28);
        return new LoanConfirmation("LN-" + System.currentTimeMillis(), memberId, bookCopyId,
dueDate);
    }

    void main() {
        var service = new BorrowBookService();
        var loan = service.processLoan("MEM-1", "CPY-99", 2, false, 3);
        System.out.println("✅ SDD ile üretilen ve şartnameye %100 uyan kayıt: " + loan);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Acil bir canlı sistem hatasında (hotfix) kodu doğrudan IDE'den düzeltip 'şartnameyi sonra güncellerim' demek metodolojinin ölüm fermanıdır. O şartname asla güncellenmez ve sonraki kod üretiminde hotfix'iniz tamamen silinir!

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Applying a hotfix directly to production code while promising to 'update the spec later' destroys the entire methodology. The specification rots immediately, and the next automated regeneration erases your hotfix without warning.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Spec-Driven Development'ta kod değil, şartname temel varlıktır; kod onun derlenmiş çıktısıdır.

• Kodu doğrudan elle değiştirmek yasaktır; tüm değişiklikler şartnameden başlayarak koda akar.

• Şartnameleri yaşayan birer sözleşme olarak korumak, projeyi gelecekte sıfır maliyetle modernize etmeyi sağlar.

🎯 Key Architectural Takeaways

• In Spec-Driven Development, the specification is the primary asset; code is its compiled output.

• Never modify code directly; all changes must cascade downstream from the specification.

• Living specifications allow entire software systems to be safely regenerated onto new tech stacks.